

22/07/2026.

Short answers - Analysis

CSA0A0A - Operating system

Name : bomdhiya

Reg No : 192521224

1. Suppose a modern operating system supports multiple users and processes with shared resources like CPU, memory and I/O devices.
 - a) Classify different types of operating systems suitable for this scenario and justify your choice.
 - b) Draw the structure of an OS showing how resources are managed.
 - c) Write a sequence of system calls required to create and execute a new process in such an environment.
2. Consider an operating system managing multiple files and devices for concurrent users.
 - a) Categorize system calls used for file and device management.
 - b) Illustrate how the OS handles file operations using a layered structure diagram.
 - c) Write a pseudo-code or C-based system call sequences to open, read and close a file.

3. Assume a real-time Operating system used in an embedded system controlling industrial Machinery.
- Identify the type of OS and explain why it is suitable for real-time application.
 - Draw a diagram representation the interaction between hardware, kernel and user processes.
 - List and explain system calls required for process synchronization in this system.
4. A distributed computing system consists of multiple interconnected computers sharing resources over a network.
- Classify the type of Operating system and compare it with centralized OS.
 - Draw the architecture of a distributed OS showing communication between nodes.
 - Write the sequences of system calls involved in inter-process communication (IPC).
5. Consider a scenario where an OS must manage process scheduling, memory allocation and device communication efficiently.
- Explain the major functions of an OS in handling these tasks.
 - Draw a block diagram representing OS components and their interactions.
 - Write system call example for memory allocation and device management.

1. a) Classification

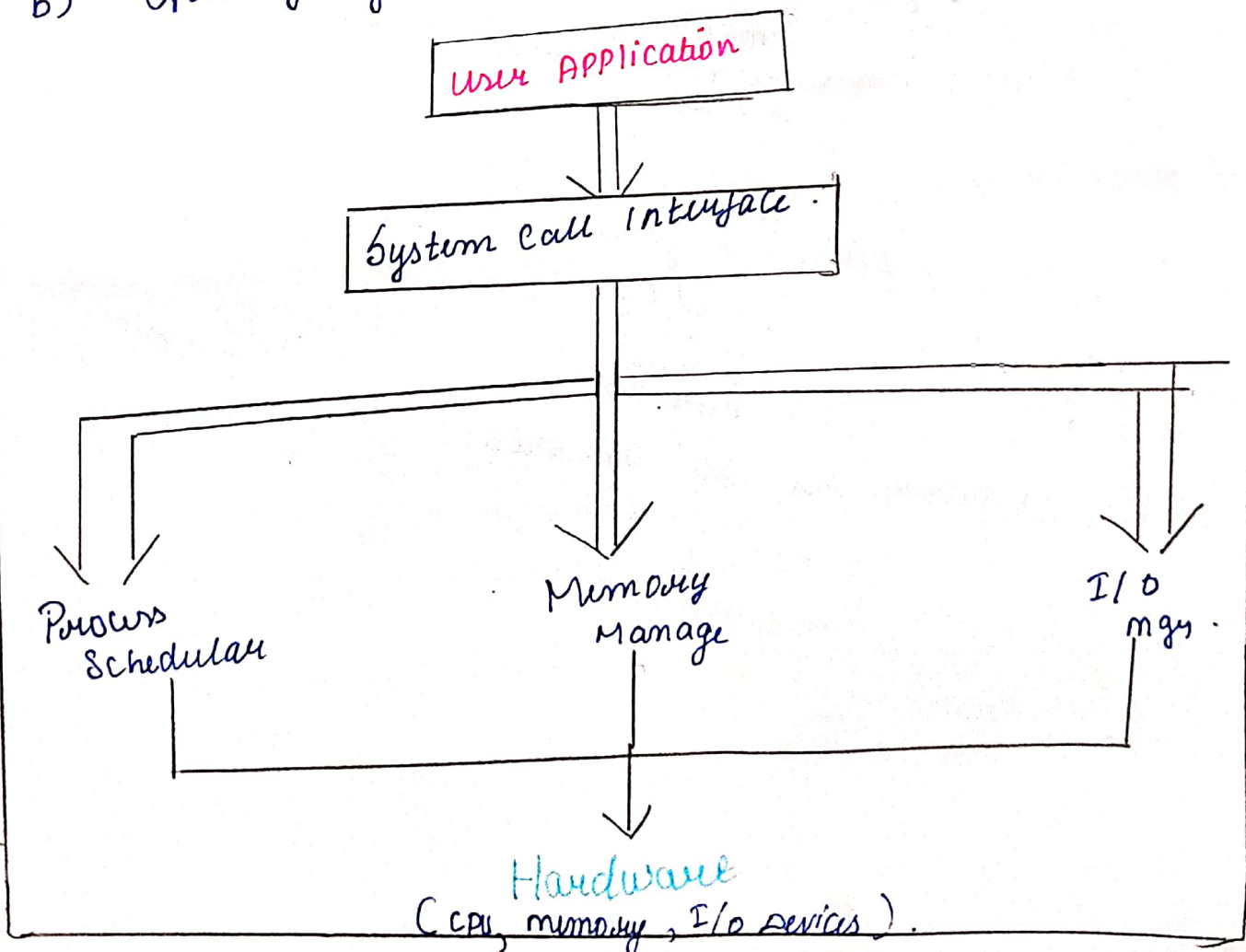
(i) multitasking operating system : which runs multiple Processes continuously .

(ii) multi-user Operating system :- which allows several users to access the system simultaneously .

(iii) Time - Sharing Operating system :- it gives each user / process a fair CPU time

∴ The best choice (Or) suitable for this scenario is Time Sharing Operating system .

b) Operating system structure .



c) P c) system call sequence to create and execute a process.

fork()
exec()
wait()
exit()

fork() → exec() → wait() → exit().

fork → creates a child process.

exec → loads new program.

wait → parent waits for child.

exit → child terminates and return to control.

2. file and device Management for concurrent users.

a) categories of system calls.

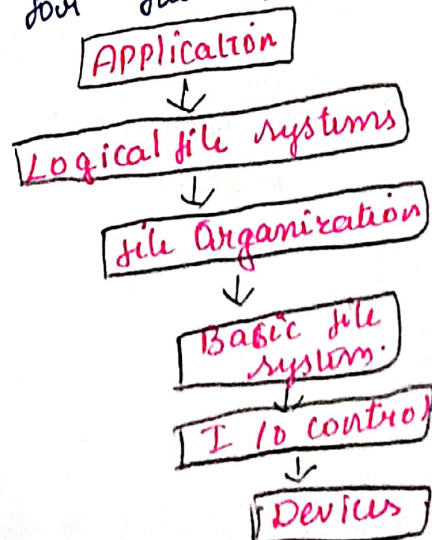
• file Management.

open()
read()
write()
close()
create()
delete().

• Device Management.

read()
write()
request()
release().

b) structure for file Operation.



c)

Pseudo code.

```
#include <fcntl.h>
#include <unistd.h>
int main() {
    int fd;
    char buffer[100];
    fd = open("data.txt", O_RDONLY);
    read(fd, buffer, sizeof(buffer));
    close(fd);
    return 0;
}
```

sequence
open() → read() → close()

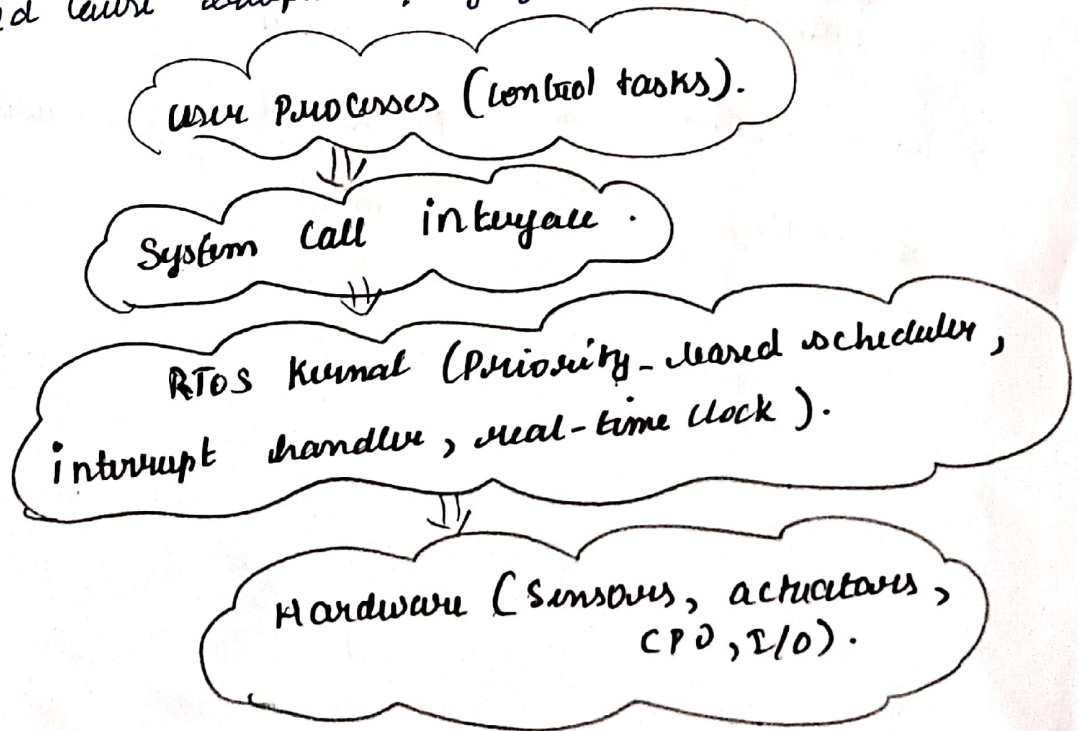
3.

a) Types of OS.

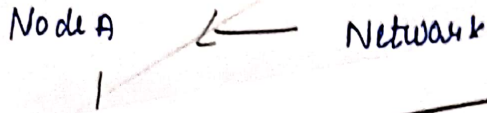
(i) Real Time Operating system (RTOS).

→ it is a Hard RTOS, since industrial Machinery control requires guaranteed response within strict deadlines, a missed deadline could cause equipment/safety failure.

b).



b).



c) systemcalls for process synchronization.

- `sem_wait()` - semaphore-based locking to prevent race conditions
- `pthread_mutex_unlock()` - mutual exclusion for shared resources.
- `signal()` - inter process signaling for event-driven synchronization.
- `sched_setscheduler()` - sets real-time scheduling policy for priority-based execution.

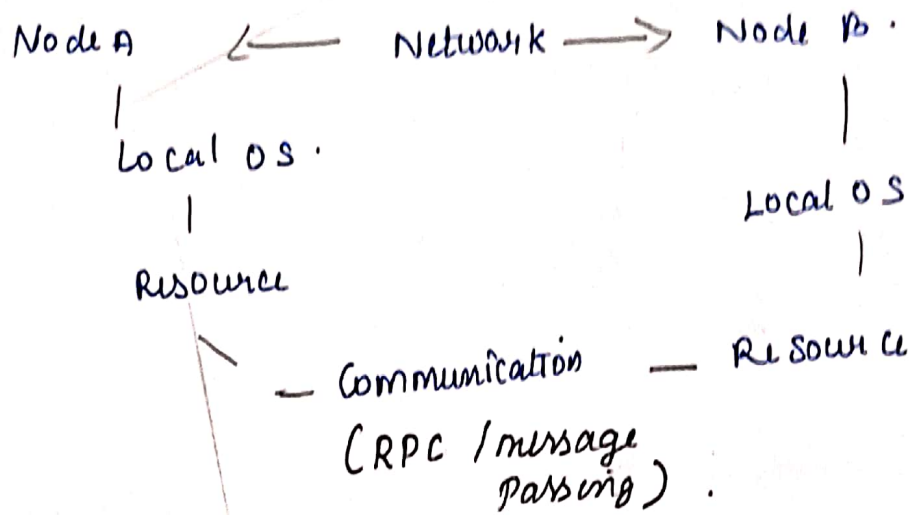
5. a

4. a)

Aspect	centralized OS	Distributed OS.
Resource location	single machine	multiple networked nodes
failure impact	Total system failure	partial
Communication	Not needed	requires message passing
Scalability	Limited.	High.



b).



c) System calls for IPC.

pipe()

msgget()

msgsnd()

msgctl()

shmget()

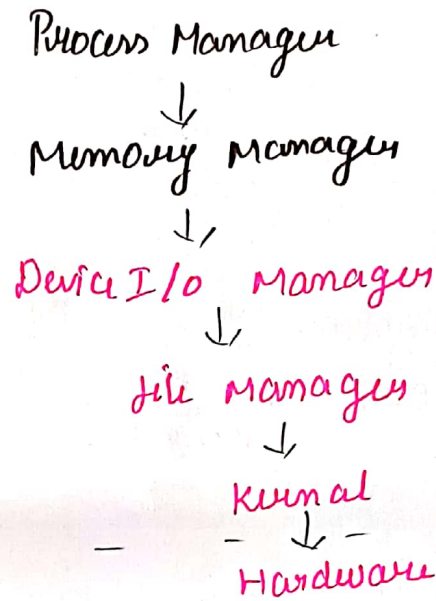
5. a) Major operating system function.

• Process Scheduling: decides which process runs next.

• memory allocation: allocates and deallocates memory to processes.

• Device Communication: Manages I/O requests through device drivers and buffering.

b) block diagram of OS components .



c) System call

- malloc () / free () // memory allocation
- free () // memory deallocation
- ioctl () // device control
- read () / write () // device I/O .